

8.0 Software Test Plan

8.1 Introduction

This section defines the Software Test Plan and Test Procedures for the Poker Face AI Web Application. The purpose of this document is to:

1. To describe what components of the system are tested and how they are tested.
2. To describe the detailed procedures used to verify that the system meets all functional and non-functional requirements.

This document follows the CMMI test documentation structure and includes:

- Unit Testing (individual modules)
- Integration Testing (combined components)
- Acceptance Testing (complete system validation)

Both positive (valid input) and negative (invalid/error input) tests are included at all levels.

The system consists of the following major Computer Software Units (CSUs):

- Game Engine Module
- Hand Evaluation Module
- AI Decision Module
- Betting Logic Module
- Web API Backend
- Database Module
- Frontend User Interface

8.2 Unit Test Plan

8.2.1 Unit Test Overview

Unit testing verifies each CSU independently to ensure correctness before integration. Testing is performed using:

- Python pytest framework
- Mock objects for database and AI dependencies
- Manual UI validation for frontend components

Each module is tested with valid inputs, invalid inputs, and boundary conditions.

8.2.2 Unit Tests Planned

Table 8-1: Unit Tests

Test ID	Module	Test Name	Inputs	Expected Output	Stub Required
UT-01	Hand Evaluator	Royal Flush Detection	Valid royal flush cards	Correct ranking returned	None

UT-02	Hand Evaluator	Invalid Card Input	Duplicate card values	Error thrown	None
UT-03	AI Decision	Valid Bet Decision	Strong hand, high stack	Raise action	Mock Game State
UT-04	AI Decision	Invalid Game State	Null hand	Exception raised	Mock Game State
UT-05	Betting Logic	Valid Raise	Bet < stack	Pot updated correctly	None
UT-06	Betting Logic	Over-Bet	Bet > stack	Error returned	None
UT-07	Game Engine	Start New Game	Valid player list	Game initialized	None
UT-08	Game Engine	Empty Player List	[]	Error returned	None
UT-09	API Endpoint	Valid Join Request	Valid JSON	200 OK	None
UT-10	API Endpoint	Invalid JSON	Missing fields	400 Bad Request	None

Both boundary testing and negative testing are included.

8.2.3 Unit Test Procedures

Procedure for Hand Evaluator Module

1. Open a terminal.
2. Navigate to the project root directory.
Run command:
`pytest server/tests/test_hand_evaluator.py`
3. Observe the test output summary.
4. Confirm:
 - Royal Flush test returns correct rank.
 - Duplicate card test raises an exception.

Procedure for AI Decision Module

1. Run:
`pytest server/tests/test_ai_decision.py`
2. Mock game state using provided fixtures.

3. Confirm:
 - Strong hand → AI raises.
 - Weak hand → AI folds.
 - Null hand → Exception raised.

Procedure for Betting Logic

1. Run:
`pytest server/tests/test_betting_logic.py`
2. Enter test values:
 - Bet = 50, Chips = 200 → Expect success.
 - Bet = 300, Chips = 200 → Expect error.
3. Confirm pot updates correctly.

8.3 Integration Test Plan

8.3.1 Integration Overview

Integration testing verifies that CSUs operate correctly when combined into Computer Software Components (CSCs). Integration is performed in the following order:

1. Game Engine + Hand Evaluator
2. (Engine + Evaluator) + Betting Logic
 - AI Module
 - Web API
 - Frontend UI

Both positive and failure conditions are tested.

8.3.2 Integration Tests Planned

Test ID	Components	Description	Input	Expected Result
IT-01	Engine + Evaluator	Evaluate dealt hands	Start round	Correct winner
IT-02	Engine + Betting	Full betting round	Valid bets	Pot correct
IT-03	Engine + AI	AI plays a full hand	Simulated game	Legal decisions
IT-04	Backend + DB	Store game result	Completed game	DB entry created
IT-05	Full Backend	API-driven round	Web request	Game state updates

IT-06	Full Stack	User plays a hand	UI interaction	Correct UI updates
-------	------------	-------------------	----------------	--------------------

8.3.3 Integration Test Procedures

IT-01: Engine + Evaluator

1. Run integration suite:
`pytest tests/test_engine_eval.py`
2. Simulate two players.
3. Force known hands.
4. Confirm:
 - Winner matches the expected hand ranking.

IT-03: Engine + AI Integration

1. Run:
`pytest tests/test_ai_gameplay.py`
2. Simulate 100 hands.
3. Confirm:
 - No illegal moves.
 - No crashes.
 - Pot resolves correctly.

IT-06: Full Stack Test

1. Start the backend server.
2. Start frontend.
3. Open the browser to localhost.
4. Create a new room.
5. Join as a player.
6. Play a full hand.
7. Confirm:
 - Bets update visually.
 - AI responds.
 - Winner displayed.
 - No UI freezes.

8.4 Acceptance Test Plan

8.4.1 Acceptance Overview

Acceptance testing validates that the complete Poker AI Web Application satisfies all customer requirements as defined in the Software Requirements Specification (SRS).

Testing is performed in a demonstration-style environment simulating real users.

8.4.2 Acceptance Tests Planned

Test ID	Requirement	Description	Expected Result
AT-01	Multiplayer Game	Two users join	Both appear at the table
AT-02	AI Opponent	AI joins	AI plays legally
AT-03	Betting Rules	Raise, Call, Fold	Correct pot and state
AT-04	Winner Detection	Complete showdown	Correct winner
AT-05	Invalid Action	Player over-bets	Error displayed
AT-06	Game Persistence	Refresh page	Game state restored

8.4.3 Final Acceptance Test Procedures

Full System Demonstration

1. Deploy the application to the staging server.
2. Open the application in two separate browsers.
3. Create a room.
4. Join from a second browser.
5. Play a complete hand.
6. Force showdown.
7. Verify:
 - Correct winner.
 - Pot distribution is correct.
 - No crashes.
8. Attempt invalid action.
9. Confirm that an error message is displayed.
10. End session.

Acceptance criteria: All tests pass without system failure.

8.5 Test Configuration Control

All test scripts are stored in the /tests directory within the Git repository.

- Unit tests: /tests/unit
- Integration tests: /tests/integration
- Acceptance test scripts: /tests/acceptance

Continuous Integration (CI) pipeline automatically runs:

- All unit tests on every commit.
- Integration tests on merge to the main branch.
- Acceptance tests before release builds.

All tests are version-controlled alongside production code. The build process ensures all dependencies are locked via requirements.txt and package.json.

8.6 Items Not Tested

The following items are not tested:

1. Third-party library internal implementations (e.g., FastAPI internals).
 - Rationale: Assumed verified by vendor.
2. Network latency under real-world internet variability.
 - Rationale: Out of project scope.
3. Security penetration testing.
 - Rationale: Not required for MVP scope.

8.7 Test Verification Matrix

Requirement ID	Requirement Description	Test ID
R-01	The system shall allow players to join the game	AT-01
R-02	System shall support an AI opponent	AT-02
R-03	The system shall enforce poker betting rules	UT-05, AT-03
R-04	The system shall evaluate winning hands	UT-01, IT-01
R-05	System shall reject invalid bets	UT-06, AT-05
R-06	System shall persist game results	IT-04, AT-06